REPUBLIC OF BENIN

****

MINISTRY OF HIGHER EDUCATION AND SCIENTIFIC RESEARCH

****

UNIVERSITY OF ABOMEY-CALAVI

****

ÉCOLE POLYTECHNIQUE D'ABOMEY-CALAVI

****

DEPARTMENT OF ELECTRICAL ENGINEERING – INDUSTRIAL PROCESS CONTROL

*****

4th Year

INTERSHIP REPORT

REAL-TIME MONITORING SOFTWARE FOR A MOTOR-ALTERNATOR TEST BENCH

Conducted by:
ADIKPETO Peace G.

Under the supervision of
Eng. ZINSOU Aimé

Academic Year: 2025–2026

ACKNOWLEDGEMENTS

At the outset, we must thank God, the Almighty, for all the grace bestowed upon us. He sustained us, provided for our health and various needs throughout this internship, and allowed everything to be successfully completed. Let us also formally distinguish and thank specific individuals who supported us along the way. Our sincere gratitude goes to:

- **Dr. CHETANGNY Patrice Koffi**, Lecturer-Researcher, Ph.D. in Electrical Engineering, and Head of the Electrical Engineering Department at EPAC;
- The entire faculty of the Electrical Engineering Department at EPAC;
- **Eng. ZINSOU Aimé**, Chief Executive Officer of **ORANGE TECHNOLOGIES SARL**;
- Mr. ZINSOU Fernand, for his invaluable advice and technical expertise;
- Mrs. YELINDO Feryel, for her steadfast support and guidance;
- Mrs. KOUTCHIKA Nelly, for her dedicated accompaniment;
- Our dear parents, for their continuous support, wise advice, and financial assistance.

1. Introduction

1.1 Internship Context

This report documents the internship completed as a mandatory component of the Electrical Engineering – Industrial Process Control (4th year) curriculum at the École Polytechnique d'Abomey-Calavi (EPAC). Building directly upon our coursework in embedded systems and industrial instrumentation, the ultimate objective of this internship was to design a real-time supervision software suite for a motor-alternator test bench. The internship took place from August 10 to September 4, 2026, at ORANGE TECHNOLOGIES.

The methodology adopted throughout this period followed a strict logic of progressive skills development before tackling the core of the project. Each preliminary step allowed us to master a specific technical subset (serial communication, real-time graphical plotting) using simple, risk-free hardware before combining and scaling these mechanisms up to the industrial Modbus RTU protocol on the actual physical test bench.

1.2 Host Company Overview

The internship was hosted by ORANGE TECHNOLOGIES, located in Tanto, near the Tanto Primary School, Akpakpa, Cotonou. The company specializes in electronics and provides engineering analysis, design, and maintenance services across multiple fields: electronics, IT, solar photovoltaic systems, and building electrical engineering. It features a fully equipped testing workshop as well as a specialized electronics laboratory.

1.3 Internship Objectives

- Design and develop a Qt 6 / C++ supervision application for an industrial motor-alternator test bench communicating via Modbus RTU over a physical RS-485 link.
- Implement a robust, extensible, and internationalized software architecture, including measurement data persistence and automated telemetry export functionalities.

1.4 Software Application and Use Case

This real-time monitoring software allows operators to visualize critical data from the motor-alternator group simultaneously: current, voltage, torque, rotational speed, as well as mechanical and electrical power. The cross-analysis of these time-series parameters serves to evaluate their mutual impact, providing precise engineering insights for optimizing the sizing of a motor required to drive a given alternator architecture.

1.5 Methodological Approach

The project was structured into three sequential phases, each building directly on the technical milestones validated in the previous step:

- A first preliminary phase dedicated to controlling an SG90 servomoteur from a Qt application, for the explicit purpose of mastering the fundamental mechanisms of QSerialPort (port initialization, command transmission, asynchronous acknowledgement handling).
- A second preliminary phase focused on driving a stepper motor and acquiring multiple physical quantities (current, temperature, speed, torque) to master multi-curve, multi-axis plotting workflows using QCustomPlot.
- The main phase of the project, which consisted of transposing and scaling these validated software architectures to handle the real-world supervision of an instrumented motor-alternator test bench communicating via Modbus RTU over a multi-channel RS-485 link.

2. Tools, Protocols, and Hardware Overview

2.1 Development Environment: Qt / C++

Qt is a cross-platform C++ application development framework widely utilized across embedded systems and industrial automation. It was selected for this project because it consolidates serial communication modules (Qt Serial Port), dedicated industrial fieldbus modules (Qt Serial Bus), and an event-driven Signals/Slots architecture within a single

ecosystem. This asynchronous mechanism is indispensable for communicating with external hardware devices without locking the Graphical User Interface (GUI) thread while awaiting responses.

2.2 The RS-485 Communication Link

RS-485 is a serial communication standard defining the electrical characteristics of the physical layer. Unlike standard single-ended signaling referenced to ground (such as RS-232), RS-485 implements a differential signaling pair (A and B), rendering it highly resilient to electromagnetic interference (EMI) typical of industrial environments. It supports multi-point multidrop topologies, allowing up to 32 nodes on a single bus over distances reaching several hundred meters. This extreme robustness justifies its systematic use in industrial instrumentation and its selection for linking the supervision PC to the test bench sensors.

2.3 The Modbus RTU Protocol

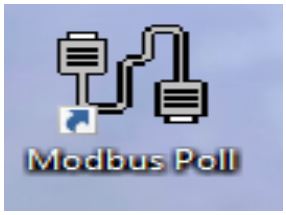
Modbus RTU is an application-layer protocol carried physically over the RS-485 link. It operates on a strict Master/Slave architecture: only the master (the monitoring application) initiates queries, while each slave device (sensor, meter) responds exclusively when addressed via its unique bus ID. Data points are organized into 16-bit numbered registers, accessed by the master using standard function codes (e.g., Function Code 03 to Read Holding Registers, Function Code 04 to Read Input Registers). Every packet exchanged is protected by a cyclic redundancy check (CRC16) checksum, ensuring the deterministic detection of any data corruption caused by electrical noise.

2.4 Diagnostic Tools: Modbus Poll and DockLight

Two independent, generic software utilities were deployed alongside our custom application to diagnose and troubleshoot hardware units that initially failed to respond:

- **Modbus Poll:** A commercial GUI utility providing a generic Modbus master simulation to manually query hardware registers without writing code. This allowed us to verify hardware-

level responsiveness independently from our software stack, isolating code bugs from physical hardware misconfigurations.



- **DockLight:** A low-level serial terminal utility used to monitor raw hexadecimal bytes exchanged over the COM port. This verified physical layer transmission and reception, completely independent of the Modbus protocol parsing layer.



2.5 Test Bench Sensors and Instrumentation

Three primary industrial measurement devices were networked onto the RS-485 bus via an interfacing module:

- **GTS208 (GALOCE):** A non-contact dynamic torque sensor inline-mounted directly between the motor and alternator shafts. It measures torque (N·m), rotational speed (RPM), and mechanical power, exposing these values natively in real time over Modbus RTU.



- **EM-100D-DIN**: A three-phase industrial electrical energy meter communicating over Modbus RTU to instrument the electrical output side of the test bench.



- **PZEM-017**: A DC electrical parameter measurement module (voltage, current, power, accumulated energy) utilizing Modbus RTU, commonly deployed for lower-power testing configurations.



- **Waveshare USB TO 4CH RS485**: An industrial-grade converter that expands a single USB port into 4 independent, concurrent RS-485 channels. Each channel operates autonomously, supporting up to 32 nodes without cross-channel interference or bus collisions, with transmission speeds from 1,200 bps to 460,800 bps over distances up to 1.2 km.



3. Preliminary Phase A — Mastering QSerialPort via Servomotor Control

This phase aimed to master asynchronous serial communication mechanisms under Qt on a highly simplified use case: controlling the angular positioning of an SG90 servomotor from a Qt GUI, using an Arduino Uno as the physical hardware intermediary.

3.2 Hardware Utilized

Le layout paired an Arduino Uno with an SG90 servomotor connected directly to a PWM pin, 5V rail, and ground. No RS-485 hardware was used at this stage; communication between the PC and the Arduino was handled via direct USB-Serial emulation to isolate and test the core QSerialPort mechanisms.

3.3 Custom Communication Protocol Design

A lightweight, human-readable ASCII text protocol was designed to allow manual verification via the serial monitor before Qt integration. An angle command is transmitted in the format A090\n (three zero-padded digits), and the Arduino transmits back an acknowledgement string OK:090\n once the target position is reached. This text format decoupled firmware parsing errors from Qt application bugs during debugging.

3.4 Arduino Firmware

The firmware utilizes the standard Servo library. Inbound serial data is read character-by-character, accumulated into a buffer until a newline character (\n) is detected, and parsed to

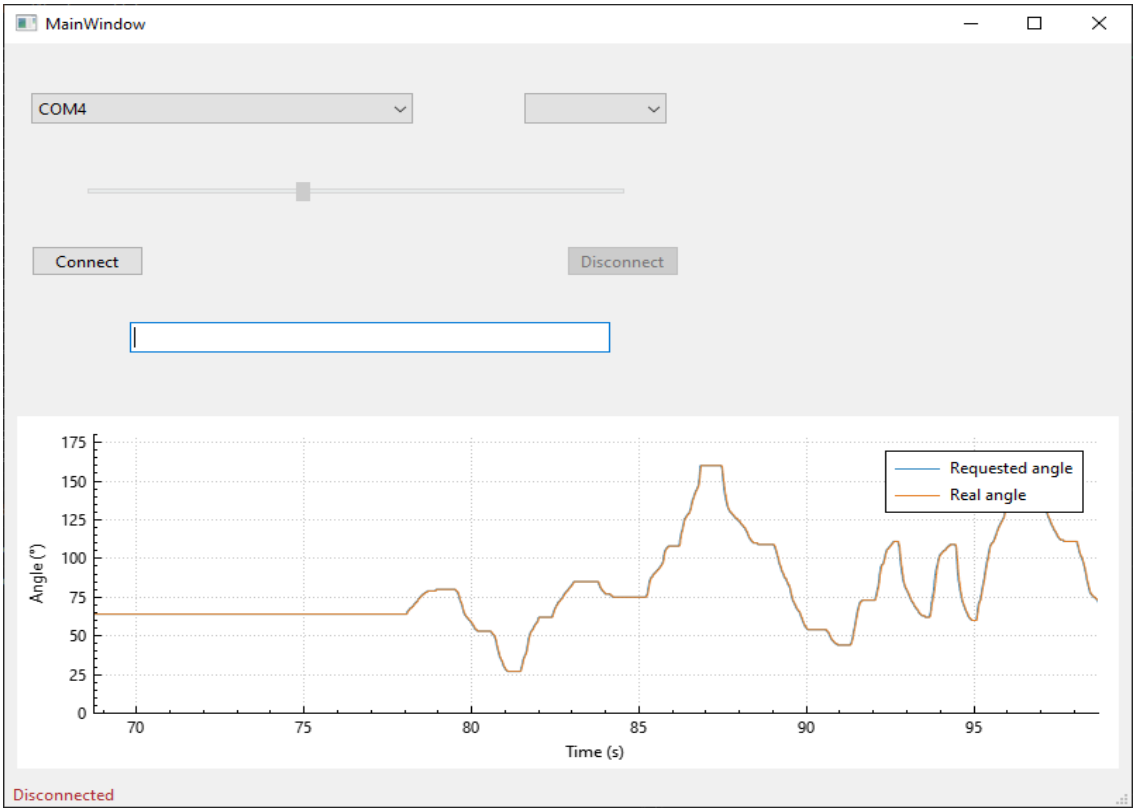
extract the numerical angle. Boundary checking (0° to 180°) is strictly enforced before updating the physical servo position.

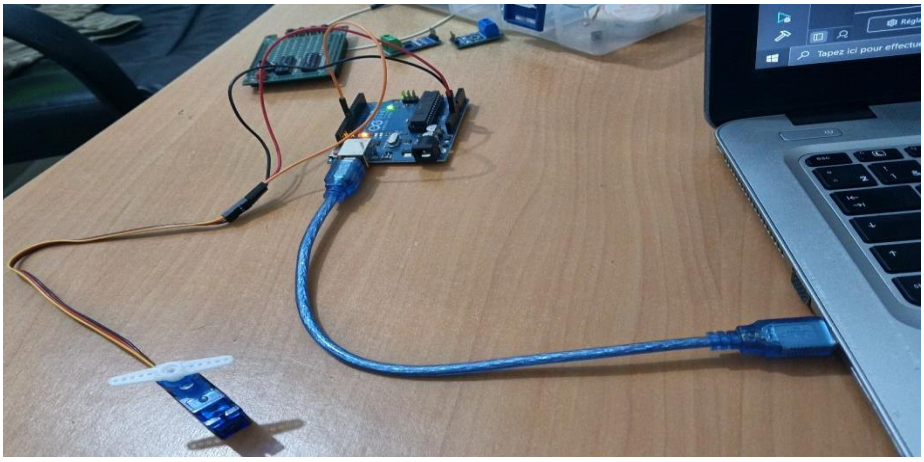
3.5 Qt Control Application

- Explicit initialization and configuration of the serial port connection parameters (baud rate, data bits, parity, stop bits) via the QSerialPort class.
- Command transmission triggered dynamically by slider (QSlider) movements, formatted to fit our custom ASCII protocol.
- Asynchronous data reception driven by the readyRead() signal, implementing a local FIFO ring buffer and enforcing complete line isolation via canReadLine().
- Real-time plotting using QCustomPlot to map targeted angles against confirmed physical angles over time, visually capturing the serial link propagation delay and physical motor response.

3.6 Results and Key Takeaways

This phase fully validated our asynchronous read/write pipeline under QSerialPort and proved the necessity of robust stream buffer management to prevent partial packet fragmentation bugs. The validated architecture (port management, asynchronous event hooks, frame boundary detection) was directly scaled up and migrated into the subsequent, more complex phases of the project.





4. Preliminary Phase B — Mastering QCustomPlot via Stepper Motor Control

4.1 Objective of this Step

This phase focused on implementing multi-curve plotting routines featuring independent, multiple vertical Y-axes within QCustomPlot. This allowed the simultaneous real-time rendering of distinct physical quantities characterized by highly disparate scales and engineering units.

4.2 Stepper Motor Selection

An initial trial using a 5V PWM-driven DC motor was superseded by a NEMA17 stepper motor driven by an A4988 controller. This change was motivated by a key technical advantage: unlike a brushed DC motor, a stepper motor yields deterministic velocity data directly calculated from the microcontroller's step pulse train frequency (assuming no stall condition), eliminating the need for an external tachometer sensor and simplifying the physical setup while preserving the multi-variable data tracking objective.

4.3 Acquired Quantities and Associated Sensors

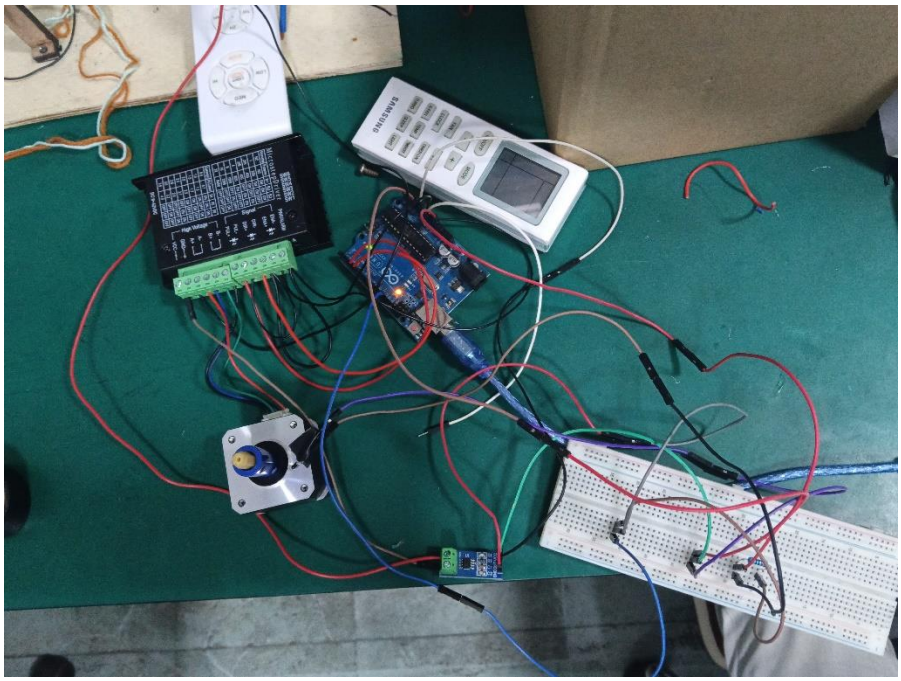
- The target step velocity profile, controlled dynamically from the Qt GUI (open-loop).
- Physical motor current consumption, monitored via an ACS712 Hall-effect current sensor.
- Thermal variations, captured via a 100k NTC thermistor.
- Output torque values, mathematically derived in real time from the measured current and the motor's native torque constant ($T = Kt * I$), bypassing the need for a physical torque transducer at this preliminary stage.

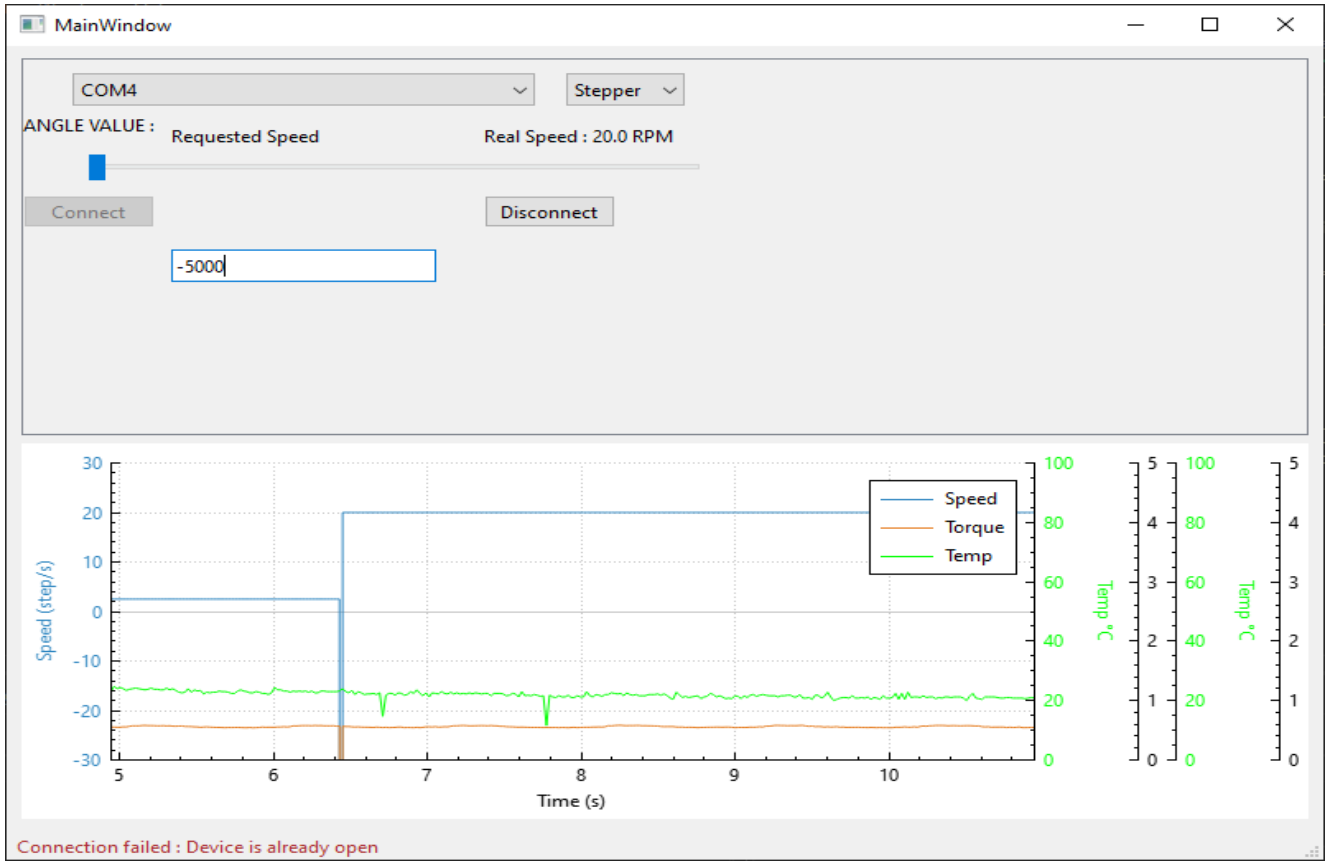
4.4 Multi-Axis Visualization with QCustomPlot

The core technical challenge resided in mapping widely varying numerical scales simultaneously onto a single timeline (velocity in steps/sec, current in Amperes, temperature in °C, and torque in N·m). To avoid lower-amplitude curves being visually crushed or flattened by larger scales, a multi-Y-axis object architecture was deployed. Each variable was allocated an independent vertical axis, custom scaling properties, and a unique tracking color.

4.5 Results and Key Takeaways

This phase validated the dynamic construction of multiple asynchronous coordinate axes within a single plot canvas without degrading UI rendering fluidities or drop frames. This validated visualization backbone formed the basis of the primary phase of the project, where the telemetry load was significantly higher.




MainWindow
COM4
Stepper
ANGLE VALUE :
Requested Speed
Real Speed : 20.0 RPM
Connect
Disconnect
-5000
Speed
Torque
Temp
Speed (step/s)
Temp °C
Time (s)
Connection failed : Device is already open

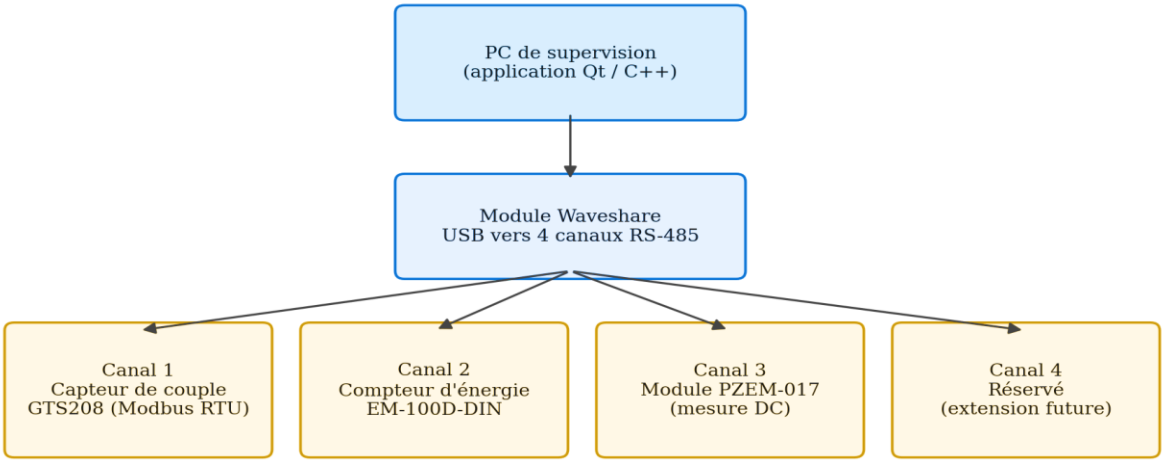
5. Core Project — Modbus RTU Supervision of the Motor-Alternator Test Bench

5.1 Test Bench Overview

The main phase focused on an industrial test bench coupling an electric motor to an alternator through an inline dynamic torque sensor. The primary objective was to evaluate the load behavior of the alternator against varying motor profiles through the concurrent acquisition of torque, speed, mechanical power, and input electrical variables.

5.2 Hardware Architecture Selection

Unlike the preliminary phases, no intermediate microcontroller abstracting data collection was used. Every field device dialogues directly with the supervision PC in Modbus RTU via the industrial Waveshare USB-to-4CH RS-485 hub, which exposes four distinct virtual COM ports. The torque sensor deployed is a GALOCE GTS208, exposing torque, speed, and power directly inside Modbus registers.



5.3 Communication Protocol Validation for the Torque Sensor

The manufacturer documentation for the GTS208 allowed us to map out the exact register addresses, data types, and word lengths. A critical engineering problem was resolved here: by analyzing the sensor's documented internal self-test register (which outputs a fixed constant of 3.14), we mathematically deduced the exact 32-bit floating-point word order (CDAB layout).

This parameter is rarely documented explicitly by manufacturers and represents a frequent source of data decoding errors.

5.4 Development of a Modbus Device Discovery Tool

To address the difficulties of deploying unconfigured or undocumented field sensors that initially failed to return responses, a custom two-tier Modbus scanner tool was developed:

- **Physical Layer Discovery:** Automatically sweeps through combinations of baud rates, parity, stop bits, and Slave IDs until a valid frame response is caught on the bus.
- **Application Layer Discovery:** Sweeps through function codes and register blocks once physical-layer connectivity is established.

A significant optimization implemented in this utility was distinguishing between a total communication timeout (wrong physical connection settings) and a valid Modbus Exception Frame (verified CRC packet, but unsupported register or address). An exception response constitutes absolute proof that the physical link settings are correct, preventing naive diagnostic routines from logging it as a total link failure.

5.5 Supervision Software Architecture

The application enforces a strict Object-Oriented separation between a sensor's abstract blueprint (measurement channels, decoding rules, and visual style) and its volatile physical session attachment attributes (assigned COM port, baud rate, slave ID). This design pattern allowed us to implement on-the-fly hot reconfigurations of connection settings and to dynamically attach reference user manuals directly to active sensor entities without rewriting the underlying architecture.

To prevent physical bus collisions, each sensor object allocated to a session is granted its own dedicated QModbusRtuClient reference and its own isolated cyclical polling timer loop, ensuring that only one request packet is in flight at any given time on any single physical bus.

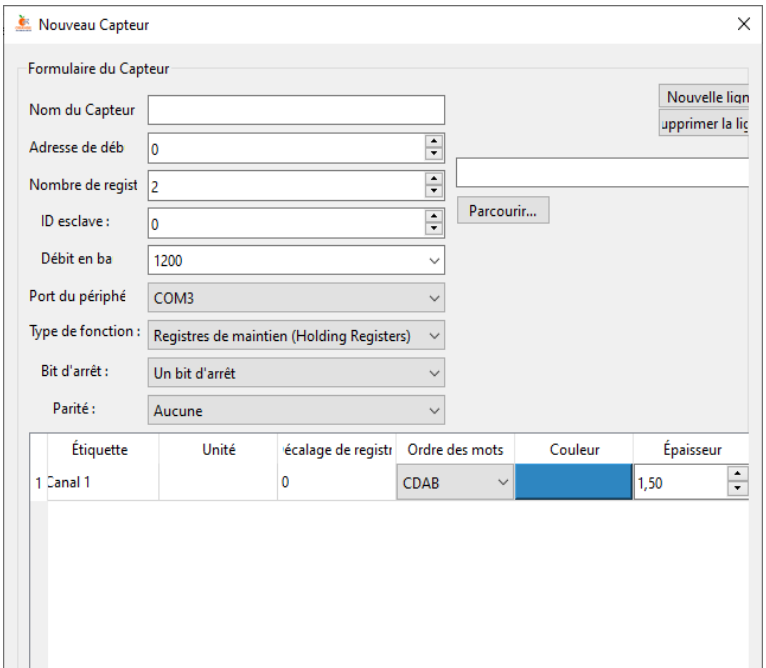
5.6 Measurement Session Management and Data Persistence

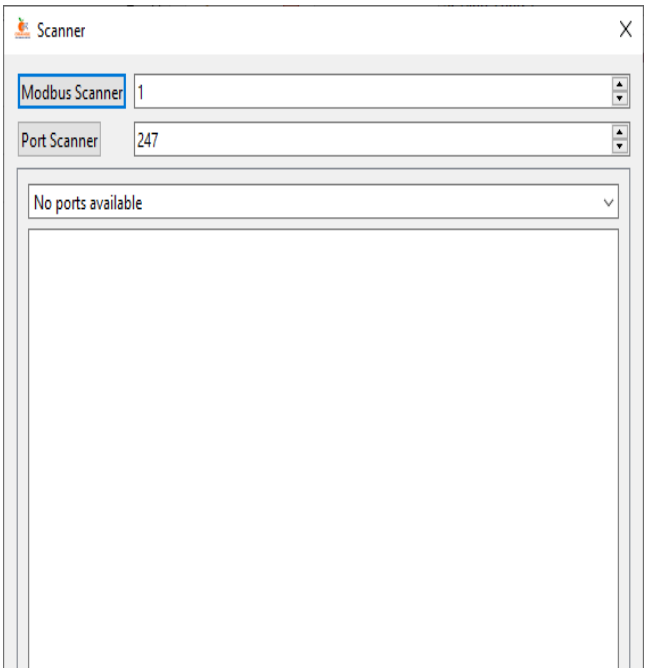
Every measurement run is mapped to an isolated, self-contained directory on disk. This directory wraps a dedicated local SQLite database file, a sensor configuration metadata manifest, copies of attached device documentations, and an export folder. This sandboxed approach ensures that any session can be safely copied, archived, or transferred independently, without relying on a shared central database engine.

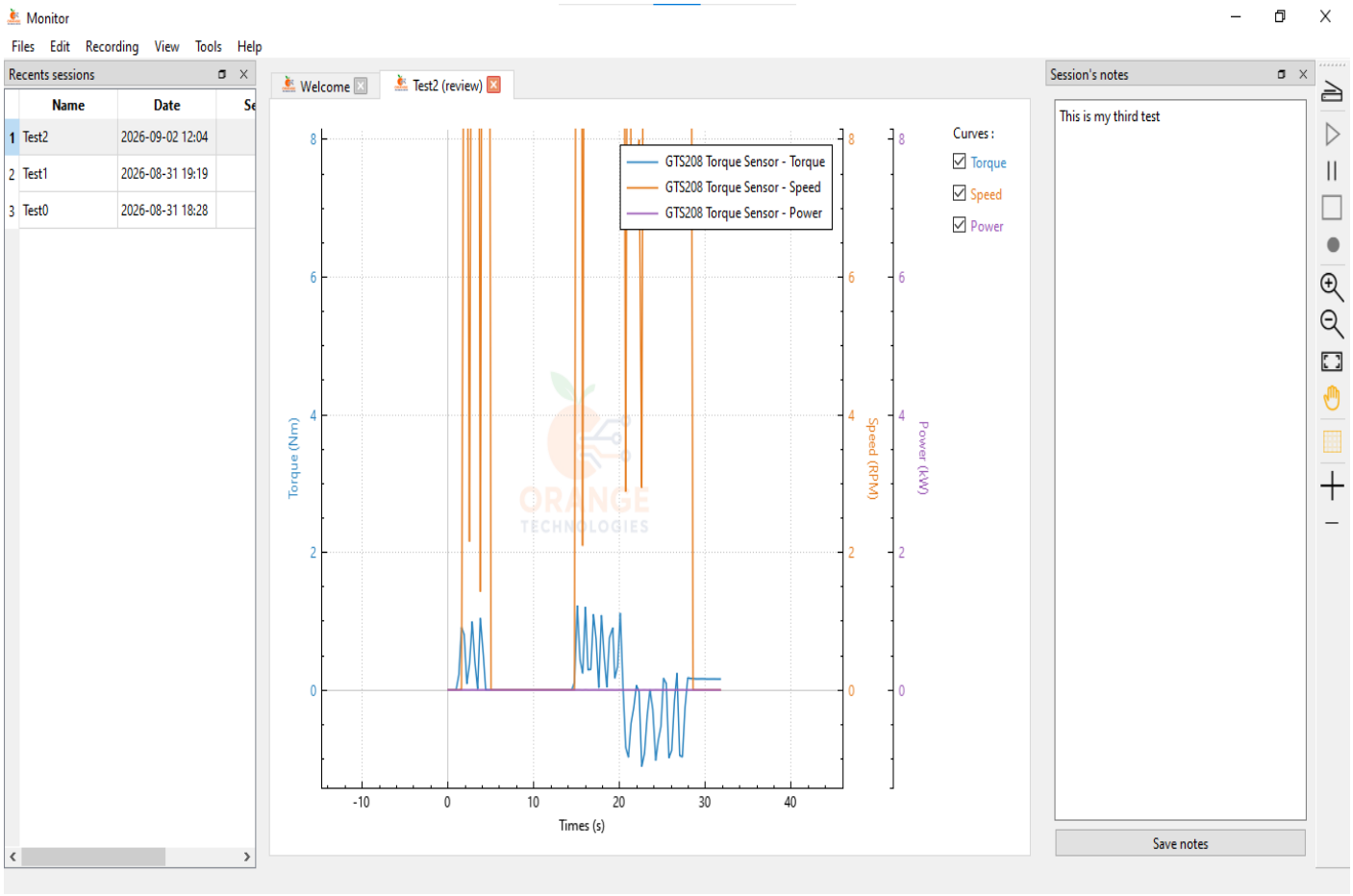
A dedicated Review Mode allows users to reopen historical data folders and reload full-scale interactive telemetry onto the plot canvas without requiring live physical hardware connections. During review, all recording controls are hard-locked to preserve data integrity.

5.7 Complementary Features

- Timestamps and on-demand exports of recorded telemetry data blocks to standard CSV spreadsheets, as well as high-resolution plot vector exports to PDF and PNG formats.
- A free-form annotations text pad mapped to each session to log environmental constraints and engineering observations.
- Complete graphic layout customization (line weights, curve colors, visibility toggles, and grid settings).
- Full application internationalization (English and French translation bundles) driven by automated system locale detection alongside a runtime language toggle requiring no application reboot.







6. Challenges Encountered and Implemented Solutions

Challenge Encountered	Implemented Solution
QCustomPlot compilation failure under MinGW compiler environments ("file too big" error).	Compiled QCustomPlot as an isolated Dynamic Link Library (DLL / Shared Object) rather than including it as raw source files within the primary project tree.

Complete lack of responsiveness from certain Modbus sensors (EM-100D-DIN, PZEM-017) despite correct physical wiring lines.	Implemented our systematic custom scanning tool to sweep connection permutations while differentiating timeouts from protocol exception frames. Parallel diagnostics were verified using independent industry standards like Modbus Poll and DockLight to isolate the hardware defect.
Undocumented 32-bit floating-point word order for the GALOCE torque sensor.	Utilized the internal fixed value of the documented self-test register (constant 3.14) to reverse-engineer and hardcode the correct byte-swapping structure.
Risk of silent decoding corruption (decoding mathematically plausible but physically incorrect sensor values).	Centralized all register parsing loops within a unified declarative conversion engine, and implemented cross-checks against the physical on-device LCD screens.
Reusing custom sensor profiles across independent testing sessions.	Enforced structural abstraction separating a sensor's mathematical identity from its active physical link properties.

7. Conclusion and Future Perspectives

This internship successfully achieved the end-to-end development of an industrial-grade supervision application using Qt 6 / C++ by following a strict progressive learning curve. We advanced from isolating single QSerialPort lines on simple hardware up to managing complex multi-axis rendering in QCustomPlot before deploying them combined on a multi-channel industrial Modbus RTU network.

Beyond mastering the technical capabilities of the Qt framework, this project highlighted the engineering importance of software architectures that anticipate volatile technical needs—specifically the benefits of abstracting device definitions from physical connection states, and the absolute necessity of empirically testing manufacturer decoding claims when dealing with incomplete data sheets.

Future iterations of the software stack will include the full integration of the remaining temperature arrays and power analyzers onto the live test bench, alongside the implementation of a centralized sensor catalog database shared across independent sessions to optimize workflow initialization speeds.